

Experiment 2

Implementation of Arrays

OBJECTIVE:

- Declare and initialize arrays in C++.
- Understand logical size and capacity.
- Perform array operations: Traverse, Search, Insert, Delete, Update.
- Handle overflow and invalid index conditions

INTRODUCTION:

An array is a linear data structure that stores elements of the same data type in contiguous memory locations. Arrays provide fast direct access using index values. However, insertion and deletion in the middle require shifting of elements. In this lab, we will implement a simple array-based list manually using:

- A fixed array
- A variable n to track the current number of elements

ARRAY DECLARATION AND ASSIGNMENT:

Array declaration requires specifying the data type and capacity. Logical size (n) keeps track of the number of elements currently stored. For example the code below stores even numbers from 60 to 88 in an Array and display 'n'.

```
#include <iostream>
using namespace std;

const int CAPACITY = 100;

int arr[CAPACITY];
int n = 0;

int main() {

    for (int val = 60; val < 89; val+=2) {
        arr[n] = val;
        n++;
    }
    cout<<"n= "<<n<<"\n";

    return 0;
}
```

(1)

TRAVERSE OPERATION

Traversal means visiting each element of the array sequentially.

Used for:

- Printing values
- Calculating sum
- Finding max/min
- Applying operations on each element

Traversal requires a loop from 0 to n-1.

```
void traverse() {
    if (n == 0) {
        cout << "Array is empty\n";
        return;
    }

    cout << "Array elements: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

(2)

SEARCH OPERATION

Search means finding the position of a value. Since array is unsorted, we use Linear Search:

- Start from index 0
- Compare each element
- Stop if found

Worst case → check all elements.

```
#include <iostream>
using namespace std;

const int CAPACITY = 100;

int arr[CAPACITY];
int n = 0;

int search(int value) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == value)
            return i;
    }

    return -1; // not found
```

```
}  
int main() {  
    int index;  
    for (int val = 60; val < 89; val+=2) {  
        arr[n] = val;  
        n++;  
    }  
    index= search(65);  
    cout<<index;  
  
    return 0;  
}  
 (3)
```

INSERT OPERATION

Insertion at end places element at index n. Insertion at index requires shifting elements to the right.

Function for insertion at the end:

```
void insertEnd(int value) {  
    if (n == CAPACITY) {  
        cout << "Overflow!";  
        return;  
    }  
    arr[n] = value;  
    n++;  
}  
 (4)
```

Function for insertion at a specific index:

```
void insertAt(int index, int value) {  
    if (index < 0 || index > n || n == CAPACITY) {  
        cout << "Invalid operation";  
        return;  
    }  
    for (int i = n; i > index; i--) {  
        arr[i] = arr[i - 1];  
    }  
    arr[index] = value;  
    n++;  
}  
 (5)
```

DELETE OPERATION

Deletion removes element and shifts remaining elements to the left.

```
void deleteAt(int index) {  
    if (index < 0 || index >= n) {  
        cout << "Invalid index";  
        return;  
    }  
}
```

```
for (int i = index; i < n - 1; i++) {  
    arr[i] = arr[i + 1];  
}  
n--;  
}
```

(6)

UPDATE OPERATION

Update modifies the value at a specific index without shifting.

```
void update(int index, int newValue) {  
    if (index < 0 || index >= n) {  
        cout << "Invalid index";  
        return;  
    }  
    arr[index] = newValue;  
}
```

(7)

LAB TASKS:

1. Run all operations described in codes 1-7 and record the output.
2. Construct the code implement the following:
 - i) Function to find maximum value.
 - ii) Function to reverse the array.
 - iii) Function to remove all occurrences of a value.